

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра квантовых информационных технологий, практической и прикладной информатики (ККИТПиПИ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №2
Визуализация и хранение данных

Студент группы ИНБО-12-23, Албахтин И.В. _____ (подпись)

Преподаватель Приходько Н.А. _____ (подпись)

Москва 2026 г.

Содержание

1 Цель и выполненные задания 3
2 Источники и диагностика данных 3
3 Столбчатая диаграмма Plotly 4
4 Круговая диаграмма Plotly 5
5 Линейные графики Matplotlib 6
6 Визуализация t-SNE 7
7 Визуализация UMAP 7
8 Время и качество проекций 8
9 Выводы и ограничения 9
Ответы на контрольные вопросы по визуализации 10
Источники и приложения по визуализации 19
10 Цель и задачи системы хранения данных 20
11 Виртуальная инфраструктура 20
12 Установка официального кластера Arenadata 21
13 Схема и импорт данных 23
14 Аналитические SQL-задания 25
15 Визуализация в Grafana 28
16 Выводы 30
17 Ответы на контрольные вопросы по хранению данных 31
18 Источники и приложения 35

1 Цель и выполненные задания

Цель работы — выбрать визуальные представления для табличных и многомерных данных, настроить оформление Plotly и Matplotlib, исследовать t-SNE и UMAP при разных параметрах и измерить время их выполнения. Для временных и категориальных графиков использован Bike Sharing; для проекций — набор рукописных цифр load_digits. Все 11 кодовых ячеек выполнены в чистом ядре отдельной VM TIABD-Albakhtin-IV без ошибок.

2 Источники и диагностика данных

Bike Sharing содержит 731 дневное наблюдение и 16 исходных столбцов за 2011–2012 годы. Источник — UCI Machine Learning Repository, H. Fanaee-T, DOI 10.24432/C5W894, CC BY 4.0. Признаки описывают дату, календарь, погодные условия и число прокатов случайных и зарегистрированных пользователей.

```
df = pd.read_csv(DATA_PATH, parse_dates=["dteday"])
df.head()
df.info()
df.isna().sum()
df.duplicated().sum()
assert (df.casual + df.registered == df.cnt).all()
```

Результаты приведены в таблице 1.

Таблица 1 – Первичная диагностика Bike Sharing

Показатель	Результат
Размер	731 × 16
Период	2011-01-01 — 2012-12-31
Пропуски и полные дубликаты	0 и 0
Всего прокатов	3 292 679
Среднее за день	4 504,35
Медиана	4 548
Максимум за день	8 714

Удаление или интерполяция строк не требуются. Дата имеет тип datetime; коды сезона и погоды дополнены категориальными подписями. Температура и ветер сохраняются в нормированной шкале. Влажность переводится в проценты по метаданным. Для season применены исходные

коды, поскольку архивное и обновлённое описания дают разные словесные расшифровки. Обработанная таблица сохранена в CSV и Parquet.

3 Столбчатая диаграмма Plotly

Данные сгруппированы по календарному месяцу; по X отложен месяц, по Y — сумма cnt. Цвет столбца зависит от этой же суммы. Использован модуль plotly.graph_objects с цветовой шкалой, чёрными границами толщиной 2, заголовком по центру размером 20, подписями осей размером 16 и метками размером 14. Угол −45° эквивалентен 315°. Высота 700 px, ширина адаптивная, сетка ivory толщиной 2 и минимальные внешние поля.

```
bar = go.Figure(go.Bar(
    x=monthly.index, y=monthly["cnt"],
    marker=dict(color=monthly["cnt"], coloraxis="coloraxis",
                line=dict(color="black", width=2))))
bar.update_layout(autosize=True, height=700,
    title=dict(x=0.5, font=dict(size=20)))
bar.update_xaxes(tickangle=-45, tickfont=dict(size=14),
    gridcolor="ivory", gridwidth=2)
```

На рисунке 1 представлен результат анализа.

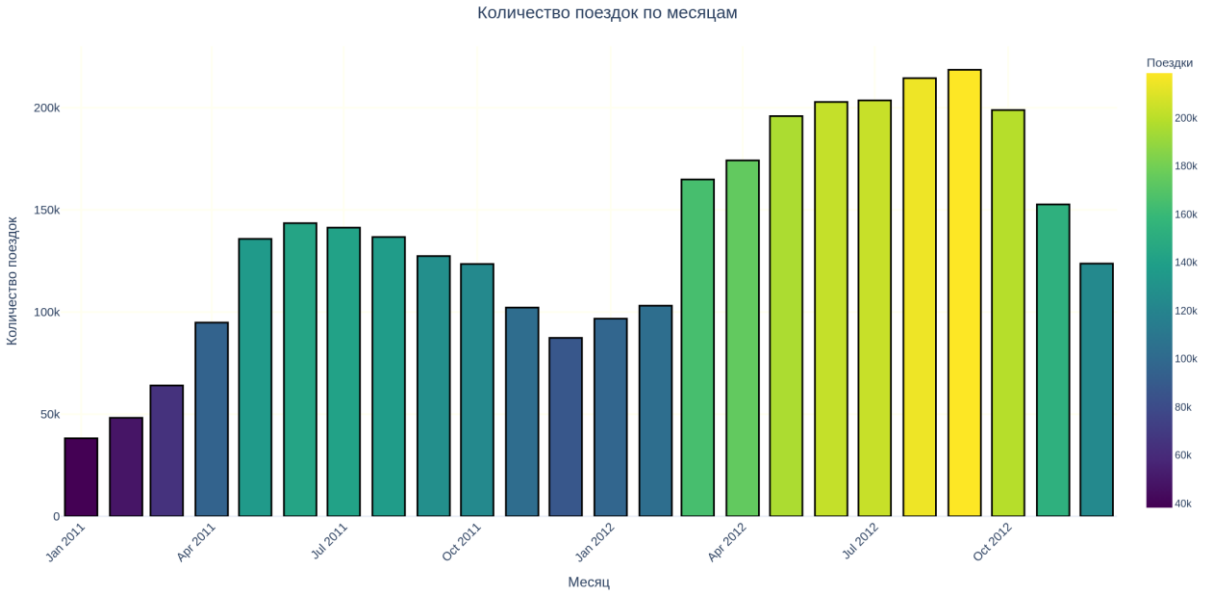


Рисунок 1 – Число прокатов по календарным месяцам

Максимум — 2012-09: 218 573 проката. Месяцы 2012 года в целом имеют больший спрос, чем одноимённые месяцы 2011 года. Цвет повторяет количественный показатель и облегчает поиск максимумов.

4 Круговая диаграмма Plotly

Для круговой диаграммы cnt просуммирован по четырём кодам сезона. go.Pie использует читаемые подписи и проценты, чёрные границы секторов толщиной 2, согласованную цветовую схему и заголовок по центру. Категорий всего четыре, поэтому дополнительное объединение мелких объектов не требуется.

На рисунке 2 представлен результат анализа.

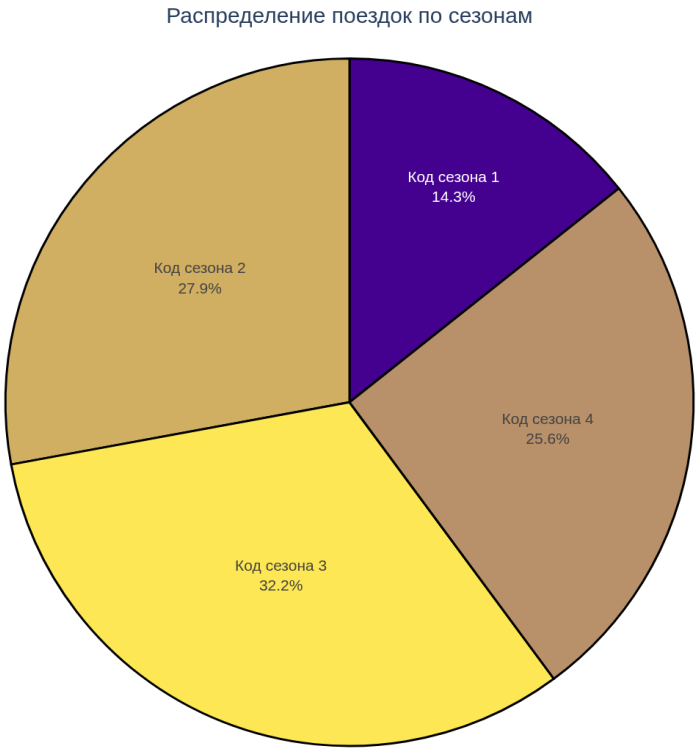


Рисунок 2 – Доли прокатов по исходным кодам сезона

Результаты приведены в таблице 2.

Таблица 2 – Количество прокатов по кодам сезона

Категория	Прокатов	Доля %
Код сезона 1	471 348	14,32
Код сезона 2	918 589	27,90
Код сезона 3	1 061 129	32,23
Код сезона 4	841 613	25,56

Максимум приходится на код 3. Словесные названия сезонов не используются для вывода, чтобы неоднозначность метаданных не превратилась в неверное предметное утверждение. Круговая диаграмма показывает структуру общей суммы, но не отражает хронологический порядок.

5 Линейные графики Matplotlib

По месяцу построены три связанных показателя: casual, registered и cnt. Каждый ряд показан отдельными осями одной Figure с линией crimson, белыми маркерами, чёрными границами маркеров толщиной 2 и сеткой mistyrose толщиной 2. Объектная модель Figure/Axes позволяет единообразно настроить подписи и шкалы.

На рисунке 3 представлен результат анализа.



Рисунок 3 – Месячная динамика трёх показателей спроса

Корреляция месячного cnt с registered равна 0,991, с casual — 0,922. Общий спрос теснее меняется вместе с зарегистрированной аудиторией. Это частично обусловлено арифметическим равенством cnt=casual+registered; показатели нельзя считать независимыми.

6 Визуализация t-SNE

Использован встроенный набор sklearn.datasets.load_digits: 1 797 изображений десяти классов размером 8 × 8 пикселей, то есть 64 числовых признака на объект. Это допустимый альтернативный готовый набор по условию задания. Признаки стандартизованы; метки применяются только для цвета точек и не участвуют в обучении проекции. Все настройки используют random_state=42.

t-SNE выполняется для perplexity=5, 30 и 50. Перплексия управляет эффективным размером локального соседства. Малое значение выделяет более мелкие группы; увеличение делает структуру менее раздробленной. Расстояния между отдельными островками и их абсолютные размеры не имеют прямого смысла исходных расстояний.

На рисунке 4 представлен результат анализа.

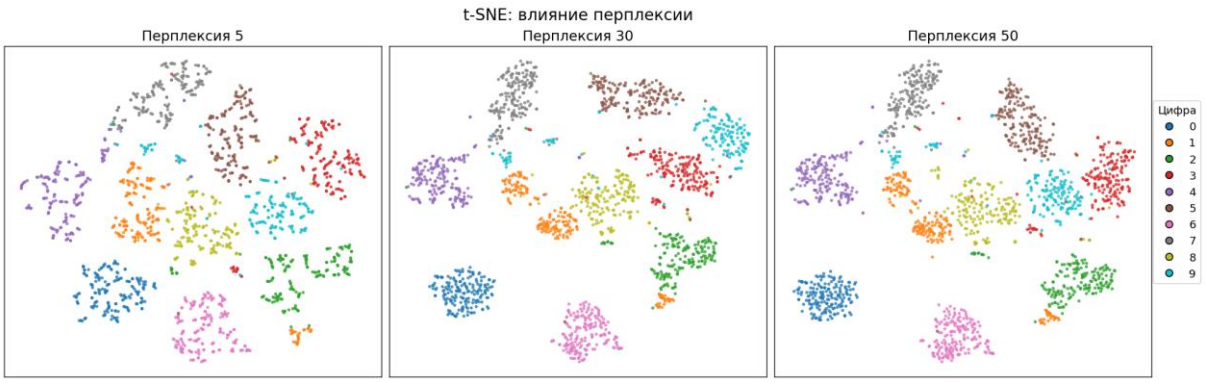


Рисунок 4 – Проекции t-SNE для трёх значений перплексии

7 Визуализация UMAP

UMAP применяется к тем же стандартизованным объектам с тремя конфигурациями: n_neighbors=5 и min_dist=0,0; 15 и 0,1; 50 и 0,5. Первая настройка подчёркивает очень локальные компактные группы; последняя оставляет больше пространства между точками и использует более широкое окружение. Сравнение проводится по одинаковому набору и цветовым меткам.

На рисунке 5 представлен результат анализа.

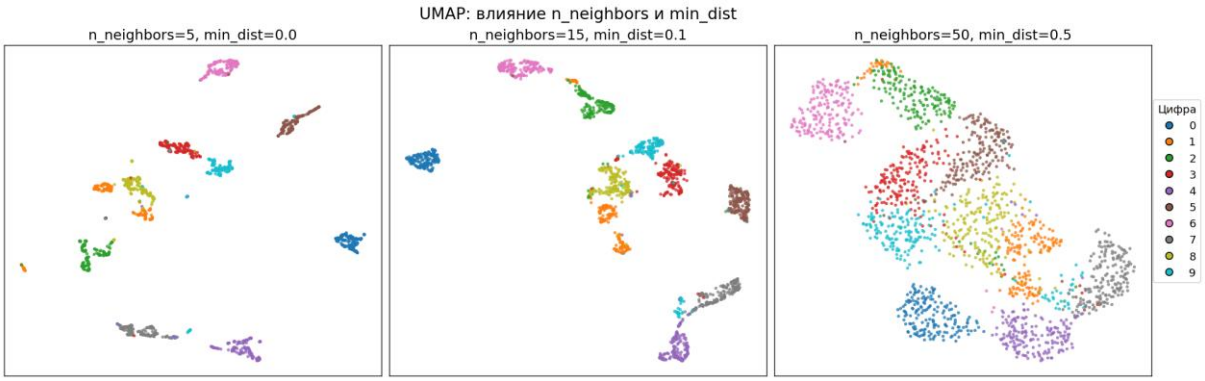


Рисунок 5 – Проекции UMAP при разных n_neighbors и min_dist

8 Время и качество проекций

Время каждого fit_transform измерено time.perf_counter(). Дополнительно рассчитана trustworthiness при k=10: она оценивает сохранение локальных соседей в двумерной проекции; выше означает лучшее сохранение этого свойства. Метрика не доказывает истинность кластеров. Первый UMAP-запуск включает JIT-компиляцию Numba, поэтому холодные и последующие измерения не полностью сопоставимы.

Результаты приведены в таблице 3.

Таблица 3 – Результаты измерения алгоритмов

Алгоритм	**Параметры**	**Время с**	**Trustworthiness**
t-SNE	perplexity=5	2,6018	0,9817
t-SNE	perplexity=30	2,7353	0,9861
t-SNE	perplexity=50	8,7063	0,9857
UMAP	n_neighbors=5, min_dist=0.0	6,2682	0,9768
UMAP	n_neighbors=15, min_dist=0.1	2,2638	0,9717
UMAP	n_neighbors=50, min_dist=0.5	4,7525	0,9468

На рисунке 6 представлен результат анализа.

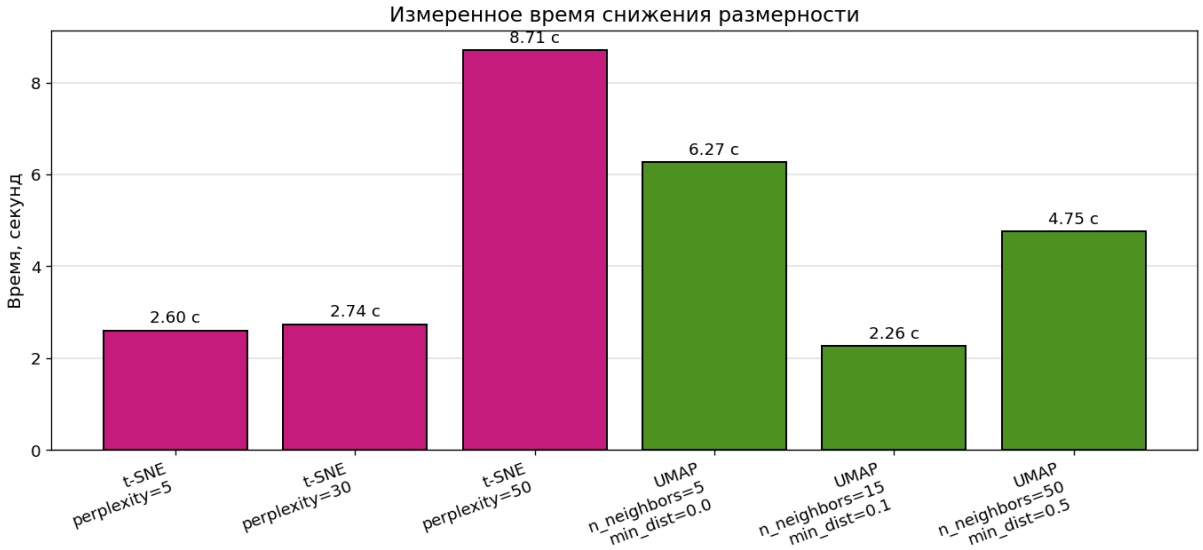


Рисунок 6 – Сопоставление времени шести конфигураций

Среднее время трёх конфигураций t-SNE — 4,6811 с, UMAP — 4,4282 с. Отношение средних 0,95 включает холодный UMAP и разные настройки. Его нельзя считать универсальным коэффициентом скорости алгоритмов.

Результаты приведены в таблице 4.

Таблица 4 – Версии окружения измерений

Компонент	**Версия**
python	3.10.12
pandas	2.2.3
numpy	2.2.6
matplotlib	3.10.8
plotly	6.3.1
scikit_learn	1.7.2
umap_learn	0.5.9.post2

9 Выводы и ограничения

1. Для количественного сравнения месяцев подходит столбчатая диаграмма; для структуры четырёх категорий — круговая; для динамики трёх показателей — линейные графики.

2. Сумма за период составляет 3 292 679 прокатов; пик — сентябрь 2012 года. Динамика позволяет одновременно увидеть колебания внутри года и общий рост.

3. Plotly добавляет интерактивные подсказки и масштабирование; Matplotlib обеспечивает точную статическую компоновку. Для каждой библиотеки выполнены требования оформления из задания.

4. t-SNE и UMAP показывают разделение известных классов цифр, но форма и плотность островков меняются при смене параметров.

5. Trustworthiness дополняет визуальный анализ измеримой оценкой локальных соседств. Она не оценивает все аспекты сохранения геометрии и не заменяет предметную интерпретацию.

6. Измерения зависят от прогрева Numba, размера набора и нагрузки. Результат одной серии на 1 797 объектах не переносится без проверки на более крупные данные.

Ограничения: Bike Sharing относится к одному городу и двум годам; исходная расшифровка сезонов неоднозначна. Двумерные нелинейные проекции теряют часть информации и не являются доказательством существования истинных кластеров.

Ответы на контрольные вопросы по визуализации

1. Из каких основных объектов состоит график Matplotlib?

Простыми словами. График Matplotlib устроен как набор вложенных объектов. Figure — весь лист или холст. Axes — отдельная область, где строятся данные. Axis — конкретная координатная ось X или Y с диапазоном, делениями и подписями. Линии, точки, текст, легенда и другие видимые элементы называются Artist.

Как это работает. Команда fig, ax = plt.subplots() создаёт Figure и один Axes. Затем ax.plot(...) добавляет линию именно в эту область. У ax есть две оси: ax.xaxis и ax.yaxis. Если написать plt.subplots(2, 2), одна Figure будет содержать четыре Axes, то есть четыре отдельных подграфика.

Пример и вывод. В практической работе рисунок с тремя временными рядами состоит из одной Figure и трёх Axes. Заголовок всего рисунка относится к Figure, а название каждого подграфика — к соответствующему

Axes. Такое разделение важно для понимания объектного интерфейса: мы изменяем конкретную часть рисунка, а не «график вообще».

2. Чем интерфейс pyplot отличается от объектного интерфейса Matplotlib?

Простыми словами. pyplot — набор команд, похожий на работу с текущим активным рисунком: plt.plot(), plt.title(), plt.xlabel(). Объектный интерфейс сначала создаёт объекты fig и ax, а затем явно вызывает их методы: ax.plot(), ax.set_title(), ax.set_xlabel().

Как это работает. pyplot хранит внутреннее состояние и решает, какой рисунок сейчас считается текущим. Для одного простого графика это удобно и коротко. При нескольких подграфиках легко случайно изменить не ту область. Объектный интерфейс не полагается на скрытое состояние: переменная ax однозначно указывает, куда добавляется линия, сетка или подпись.

Пример и вывод. В цикле можно написать for ax, column in zip(axes, columns): ax.plot(...) и настроить каждую панель отдельно. Поэтому для отчётов, сложных композиций и повторно используемых функций предпочтителен объектный интерфейс. pyplot всё равно применяется для создания Figure и быстрого показа, но основная настройка выполняется через объекты.

3. Зачем явно вызывать figure() или subplots()?

Простыми словами. Эти функции создают новый рисунок и позволяют заранее определить его размер и устройство. Без явного создания Matplotlib может использовать уже активный рисунок, из-за чего новый график случайно наложится на старый или получит неподходящий размер.

Как это работает. plt.figure(figsize=(8, 5)) создаёт Figure размером 8 на 5 дюймов, после чего pyplot работает с ней как с текущей. fig, ax = plt.subplots() одновременно создаёт Figure и Axes. Параметры nrows и ncols задают сетку подграфиков, а figsize — физический размер всего рисунка.

Пример и вывод. Для трёх рядов Bike Sharing используется plt.subplots(3, 1, ...): три области располагаются вертикально и делят общую временную ось. Явное создание делает код воспроизводимым: размер, число панелей и объект, который изменяется, видны непосредственно в программе.

4. Для чего используется zorder?

Простыми словами. zorder задаёт, какой объект рисуется поверх другого. Это похоже на слои в графическом редакторе: объект с большим значением находится ближе к зрителю, с меньшим — позади.

Как это работает. Если линия, точки и сетка занимают одно место, порядок их отрисовки влияет на читаемость. Например, сетке можно задать zorder=0, столбцам zorder=2, а подписям zorder=3. Значение не меняет координаты данных, толщину или цвет; оно влияет только на перекрытие элементов.

Пример и вывод. На scatter plot крупные точки могут закрывать линию тренда. Если линии задать больший zorder, она останется видимой. Параметр особенно полезен для фоновой сетки, выделенных наблюдений, доверительных интервалов и аннотаций. Следует задавать его осмысленно, иначе важные элементы могут оказаться скрытыми.

5. В чём различие Plotly Express и graph_objects?

Простыми словами. Plotly Express — высокоуровневый интерфейс: одна функция быстро строит типовой график из DataFrame. graph_objects — низкоуровневый интерфейс, где рисунок собирают из отдельных следов и подробно настраивают макет.

Как это работает. В Express можно написать px.bar(df, x="month", y="cnt", color="cnt"), и библиотека сама создаст Figure, следы, легенду и подписи. В graph_objects создают go.Figure(), добавляют go.Bar, go.Scatter или другие trace-объекты, а затем вызывают update_layout() и update_xaxes(). Кода больше, зато доступно больше деталей.

Пример и вывод. В этой работе go.Bar используется для явной настройки границ столбцов, цветовой шкалы, высоты и полей рисунка. Express удобен для разведочного анализа и быстрых графиков. graph_objects лучше подходит, когда нужны несколько типов следов, вторичная ось, сложные hover-подсказки или точное оформление.

6. Почему Plotly называют интерактивной библиотекой?

Простыми словами. Пользователь может взаимодействовать с готовым рисунком, а не только смотреть на статичную картинку. В браузере доступны наведение мыши, увеличение, перемещение области просмотра, включение и выключение рядов и сохранение изображения.

Как это работает. Plotly преобразует описание Figure в JavaScript-график. Данные и настройки могут быть сохранены в HTML вместе с кодом визуализации. Поэтому для просмотра такого файла не требуется запущенный Python: браузер выполняет интерактивную часть самостоятельно. В notebook график отображается через встроенный HTML/JavaScript-рендерер.

Пример и вывод. На столбчатой диаграмме можно навести курсор на месяц и увидеть точное число поездок, а затем приблизить интересующий период. Интерактивность полезна при исследовании данных. Для печатного отчёта всё равно нужна статическая версия, потому что наведение и масштабирование на бумаге недоступны.

7. Что показывает цветовая шкала в столбчатой диаграмме этой работы?

Простыми словами. Цвет каждого столбца кодирует то же значение, что и его высота: месячное число поездок. Низкие значения показаны одним краем шкалы, высокие — другим. Справа расположена легенда, связывающая цвет с числом.

Как это работает. В go.Bar массив monthly["cnt"] передан одновременно в y и в marker.color. Параметр coloraxis="coloraxis" подключает общую непрерывную шкалу. Такое кодирование называют избыточным, потому что один показатель выражается двумя визуальными каналами.

Пример и вывод. Высокий жёлтый столбец сентября 2012 года одновременно выделяется высотой и цветом. Это ускоряет поиск максимумов, но цвет не добавляет новый признак. Если цвет должен был показывать год или сезон, в marker.color нужно было бы передать соответствующую категорию и использовать дискретную легенду.

8. Почему круговую диаграмму не следует перегружать категориями?

Простыми словами. В круговой диаграмме величины сравниваются по углам и площадям секторов. Человек делает такое сравнение менее точно, чем сравнение длины столбцов. Когда секторов много, они становятся узкими, цвета повторяются, а подписи сталкиваются.

Как это работает. Круг хорошо читается, если категорий мало, их сумма действительно образует целое и различия достаточно заметны. При десяти похожих долях зритель уже не может уверенно определить порядок. Маленькие сектора почти исчезают, а вынесенные подписи занимают больше места, чем сами данные.

Пример и вывод. В практической работе всего четыре сезона, поэтому круговая диаграмма остаётся понятной и показывает долю каждого сезона в общем числе поездок. Если бы категорий было двадцать, лучше использовать отсортированную горизонтальную bar chart. Редкие категории иногда объединяют в «Другие», но только если это не скрывает важный смысл.

9. Как выбирать между Matplotlib и Plotly?

Простыми словами. Выбор определяется тем, где и как будет использоваться график. Matplotlib удобен для статических изображений и точного контроля над каждой деталью. Plotly удобен, когда пользователь должен исследовать рисунок в notebook или браузере.

Как это работает. Matplotlib хорошо экспортирует PNG, SVG и PDF, поддерживает сложные научные композиции и обычно предсказуем в печатных документах. Plotly автоматически предоставляет hover-подсказки, масштабирование, панорамирование и управление легендой. За это

приходится платить более тяжёлым HTML и зависимостью от браузерного JavaScript.

Пример и вывод. Месячные ряды в отчёте сделаны Matplotlib, потому что важна статическая трёхпанельная композиция. Столбчатая и круговая диаграммы Plotly удобны для просмотра точных значений. Часто используют обе библиотеки: Plotly — на этапе исследования, Matplotlib — для окончательной иллюстрации в отчёте.

10. Какую задачу решает снижение размерности?

Простыми словами. Объект может описываться десятками или тысячами признаков, но человек способен непосредственно увидеть только две или три координаты. Снижение размерности преобразует исходные признаки в меньшее число новых координат, стараясь сохранить важные отношения между объектами.

Как это работает. Алгоритм получает матрицу, где строки — объекты, а столбцы — признаки. На выходе он создаёт, например, две координаты для каждой строки. Разные методы сохраняют разную структуру: PCA ориентируется на общую линейную вариативность, t-SNE — на локальные соседства, UMAP — на граф соседей и сочетание локальной и более общей структуры.

Пример и вывод. Каждая рукописная цифра в load_digits описана 64 интенсивностями пикселей. t-SNE и UMAP превращают эти 64 признака в две координаты, чтобы точки можно было показать на плоскости. Проекция помогает увидеть группы и ошибки, но не хранит всю исходную информацию и не заменяет количественную проверку.

11. Что означает перплексия в t-SNE?

Простыми словами. Перплексия приблизительно задаёт, сколько ближайших соседей алгоритм учитывает при описании локального окружения каждой точки. Это не точное число соседей, но полезная интуиция для выбора масштаба анализа.

Как это работает. t-SNE преобразует расстояния между объектами в вероятности соседства. Для каждой точки подбирается ширина локального распределения так, чтобы его эффективный размер соответствовал перплексии. Малое значение сильнее ориентируется на очень близкие точки и может раздробить данные. Большое значение учитывает более широкое окружение и сглаживает мелкие детали.

Пример и вывод. В работе проверены значения 5, 30 и 50. При 5 группы распадаются на более мелкие фрагменты; 30 и 50 дают более цельные классы. Перплексия должна быть заметно меньше числа объектов, а вывод желательно проверять на нескольких значениях, потому что одна картинка может отражать настройку алгоритма, а не устойчивую структуру данных.

12. Почему для t-SNE нужно фиксировать random_state?

Простыми словами. t-SNE использует случайность при начальном размещении точек и в процессе оптимизации. Поэтому два запуска с одинаковыми данными и параметрами могут дать повёрнутые, отражённые или иначе расположенные группы.

Как это работает. random_state=42 задаёт начальное состояние генератора псевдослучайных чисел. При одинаковых версиях библиотек, данных и параметрах запуск становится повторяемым. Число 42 не обладает специальными свойствами; можно выбрать другое фиксированное целое значение.

Пример и вывод. Если сравниваются перплексии 5, 30 и 50, случайное состояние должно быть одинаковым. Иначе различия между рисунками частично объяснялись бы разной случайной инициализацией. Фиксация random_state нужна для честного эксперимента и воспроизводимого отчёта, но не гарантирует совпадение между сильно различающимися версиями библиотек или платформами.

13. Можно ли трактовать расстояния и размеры кластеров t-SNE буквально?

Простыми словами. Нет. t-SNE старается расположить близких соседей рядом, но не обязан сохранять точные расстояния между далёкими группами, их площадь, ориентацию или относительное положение на плоскости.

Как это работает. Для размещения всех точек в двух измерениях алгоритм искажает исходную геометрию. Плотность группы на картинке зависит от параметров и оптимизации. Два кластера могут выглядеть далеко друг от друга, хотя в исходном пространстве это расстояние не имеет такого масштаба. Поворот или зеркальное отражение всей проекции вообще не меняет её смысла.

Пример и вывод. На рисунке цифры 3 и 9 могут находиться рядом из-за похожих пиксельных шаблонов, но расстояние в сантиметрах между их облаками нельзя использовать как числовую меру сходства. Надёжнее анализировать локальных соседей, проверять несколько запусков и использовать метрики в исходном 64-мерном пространстве.

14. За что отвечают n_neighbors и min_dist в UMAP?

Простыми словами. n_neighbors определяет размер окружения, которое UMAP рассматривает вокруг каждой точки. min_dist задаёт, насколько близко точки могут располагаться в готовой проекции. Первый параметр управляет масштабом структуры, второй — визуальной плотностью групп.

Как это работает. При малом n_neighbors алгоритм сильнее сохраняет очень локальные отношения и может показать много небольших групп. При большом значении учитывается более широкое окружение, поэтому проявляется общая структура. Малый min_dist разрешает точкам собираться в плотные кластеры; большой оставляет между соседями больше пространства.

Пример и вывод. В конфигурации n_neighbors=5, min_dist=0.0 цифры образуют компактные островки. При n_neighbors=50, min_dist=0.5 облака становятся шире. Эти параметры не означают «правильное число кластеров»

и не выбираются только по красоте: настройки сравнивают по устойчивости, метрике trustworthiness и цели анализа.

15. Почему первое измерение времени UMAP может быть завышено?

Простыми словами. Библиотека UMAP использует Numba, которая переводит часть Python-кода в быстрый машинный код во время первого запуска. Компиляция занимает время, поэтому первый вызов включает и подготовку, и само вычисление.

Как это работает. Такой запуск называют «холодным». При следующих вызовах скомпилированные функции уже находятся в памяти или кэше, и измеряется в основном работа алгоритма. На время также влияют загрузка процессора, число потоков, размер данных, параметры и версии библиотек. Поэтому одно число без описания окружения мало что доказывает.

Пример и вывод. В работе первая конфигурация UMAP выполнялась заметно дольше двух последующих. Для честного сравнения полезно отдельно показать холодный запуск, выполнить прогрев, повторить измерение несколько раз и использовать медиану. Нельзя делать общий вывод «UMAP всегда медленнее t-SNE» по одному небольшому набору.

16. Почему двумерная визуализация не доказывает существование истинных кластеров?

Простыми словами. При переходе из 64 измерений в 2 часть информации неизбежно теряется. Алгоритм вынужден раздвигать одни точки и сближать другие. Поэтому на плоскости могут появиться убедительные островки даже тогда, когда в исходных данных нет чётких естественных границ.

Как это работает. t-SNE и UMAP специально оптимизируют вид соседств, а выбранные параметры влияют на плотность и разделение групп. Цветные метки также усиливают впечатление упорядоченности. Настоящий кластер должен подтверждаться не только картинкой, но и устойчивостью к параметрам, метриками качества, результатами других методов и предметным смыслом признаков.

Пример и вывод. Для load_digits известны правильные классы цифр, поэтому цвет помогает оценить, смешиваются ли похожие изображения. В реальной задаче меток может не быть. Тогда 2D-проекцию используют как источник гипотез: находят кандидатов на группы, а затем проверяют их в исходном пространстве и на новых данных.

Источники и приложения по визуализации

Методическое задание «Практика №2» — визуализация данных в Python, восемь пунктов задания.

Bike Sharing. UCI Machine Learning Repository. https://doi.org/10.24432/C5W894. CC BY 4.0.

sklearn.datasets.load_digits — встроенный набор рукописных цифр. Выполненный Notebook, CSV с измерениями, шесть PNG и два интерактивных HTML приложены к работе.

10 Цель и задачи системы хранения данных

Цель работы — развернуть систему хранения данных на Ubuntu Server, установить Arenadata PostgreSQL через Arenadata Cluster Manager, загрузить связанные CSV Olist в базу ecommerce, выполнить аналитические SQL-задания и представить результаты на дашборде Grafana.

Выполнены настройка виртуальной машины и сети, установка официальных бандлов SSH Common и ADPG, создание public-схемы, импорт 11 таблиц, четыре аналитических запроса, четыре обязательные и две дополнительные панели. Результаты получены на стенде 18 октября 2026 года.

11 Виртуальная инфраструктура

Результаты приведены в таблице 5.

Таблица 5 – Конфигурация виртуальной машины

Параметр	Значение
Имя ВМ	TIABD-Albakhtin-IV
ОС	Ubuntu Server 22.04.5 LTS
Ресурсы	2 vCPU; 4 ГБ RAM; диск 40 ГБ
Адаптер 1	NAT; 10.0.2.15/24; доступ в Интернет
Адаптер 2	Host-Only; 192.168.56.103/24; доступ с Windows
Хост и пользователь	tiabd-albakhtin-iv; albakhtin

На рисунке 7 показан результат выполнения на стенде.

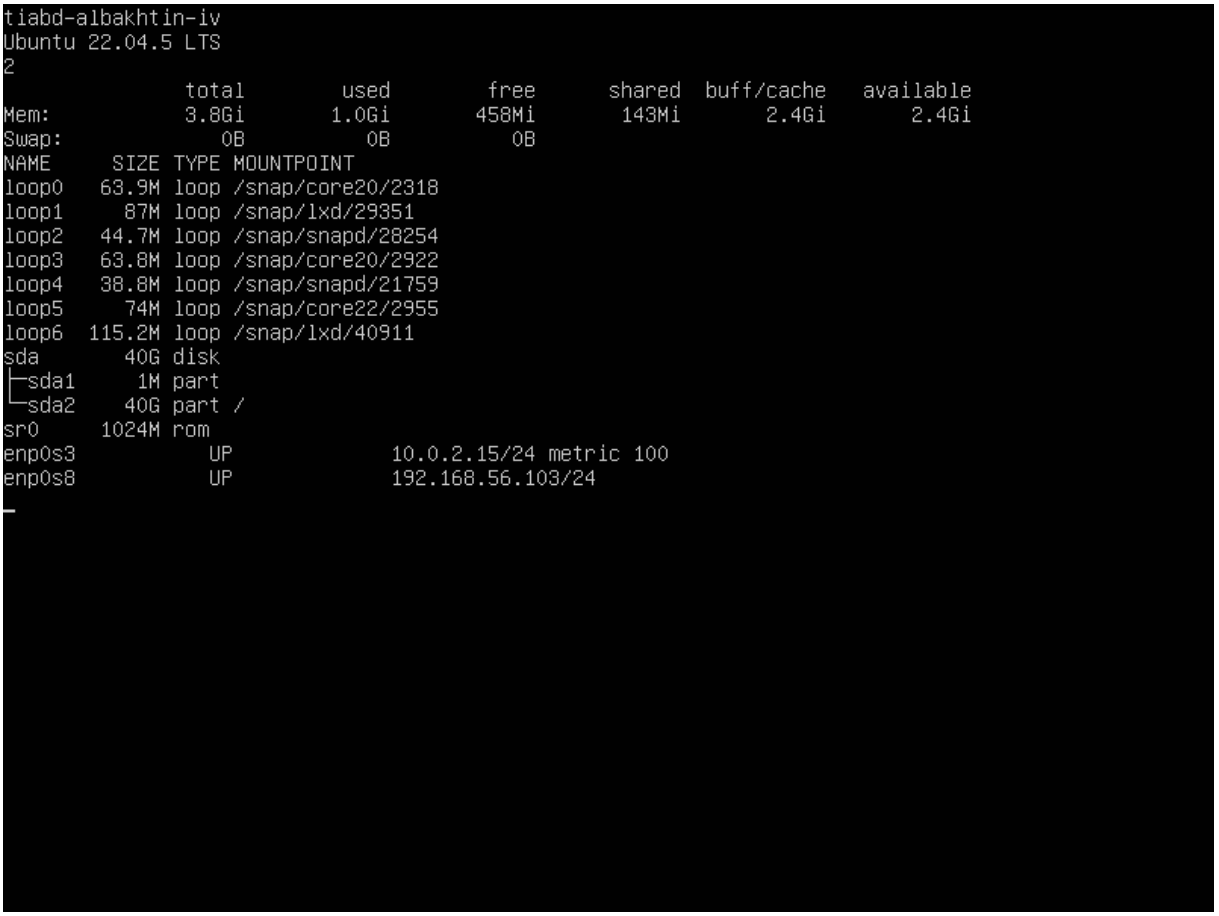


Рисунок 7 – Параметры Ubuntu Server и два сетевых адаптера

12 Установка официального кластера Arenadata

В ADCM 2.12.0 загружены официальный бандл ADPG 16.3_arenadata6.5_b2-1 Community и SSH Common 2.14-1 с downloads.arenadata.io. Созданы SSH-хостпровайдер, хост 192.168.56.103 и кластер TIABD Albakhtin IV ADPG. В выбранном бандле продукт отображается как ADP, а служба СУБД — ADPG.

На единственный учебный хост назначены ADPG, NTP Master, Prometheus Server, Grafana, Node Exporter и ADPG Exporters. Действие Install выполнено через ADCM и завершилось со статусом success. ADCM и его служебная база работают в Docker; ADPG и мониторинг установлены официальным бандлом как системные службы Ubuntu.

Результаты приведены в таблице 6.

Таблица 6 – Установленные пакеты Arenadata

Пакет	Фактическая версия
admprom-grafana	9.5.8-121+gitf31caed
admprom-prometheus	2.53.3-121+git1491d29
adpg16	16.3-arenadata1-102+git05ffe93
adpg16-pgwatch2	1.14.0-arenadata1-222+git7fa4679
adpg16-server	16.3-arenadata1-102+git05ffe93

На рисунке 8 показан результат выполнения на стенде.

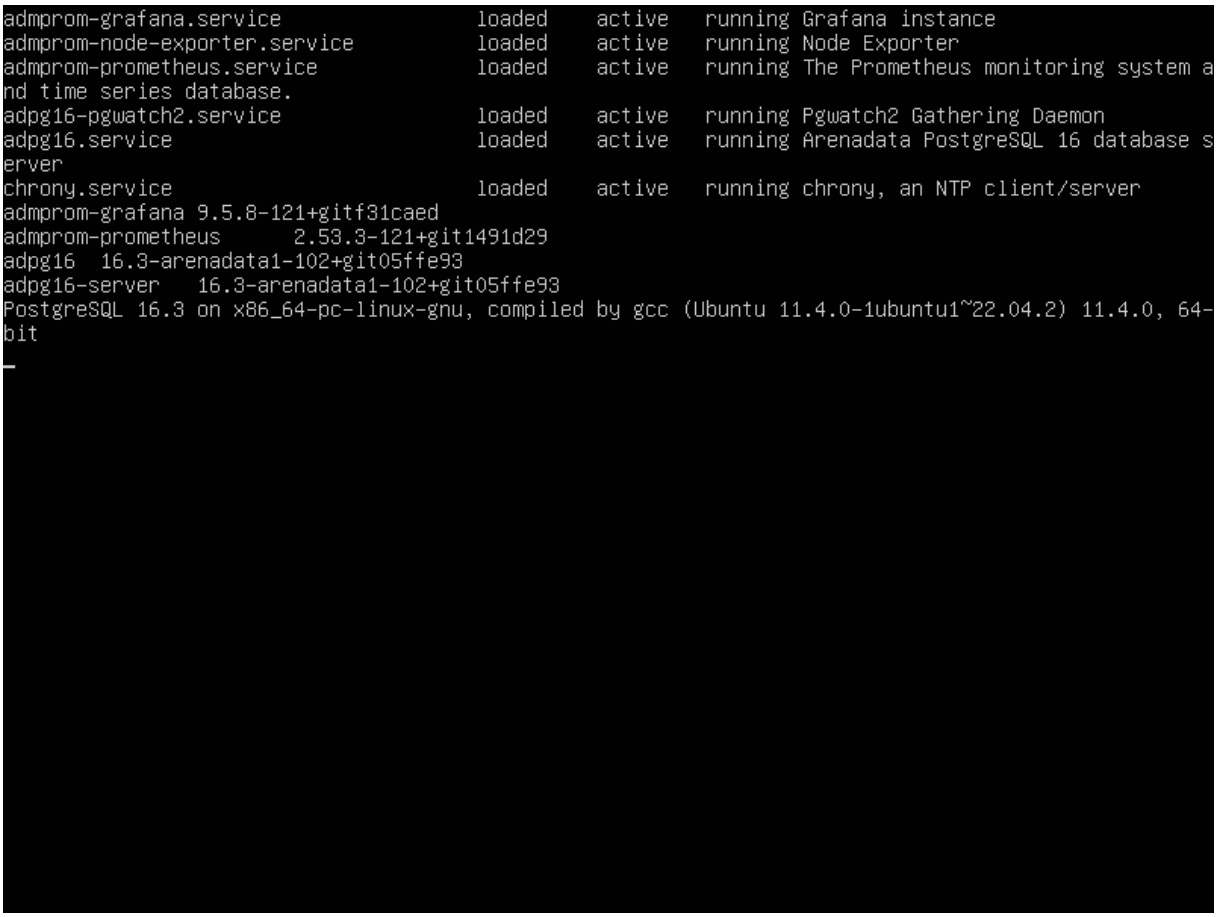


Рисунок 8 – Службы ADPG и мониторинга и версии пакетов Arenadata

На рисунке 9 показан результат выполнения на стенде.

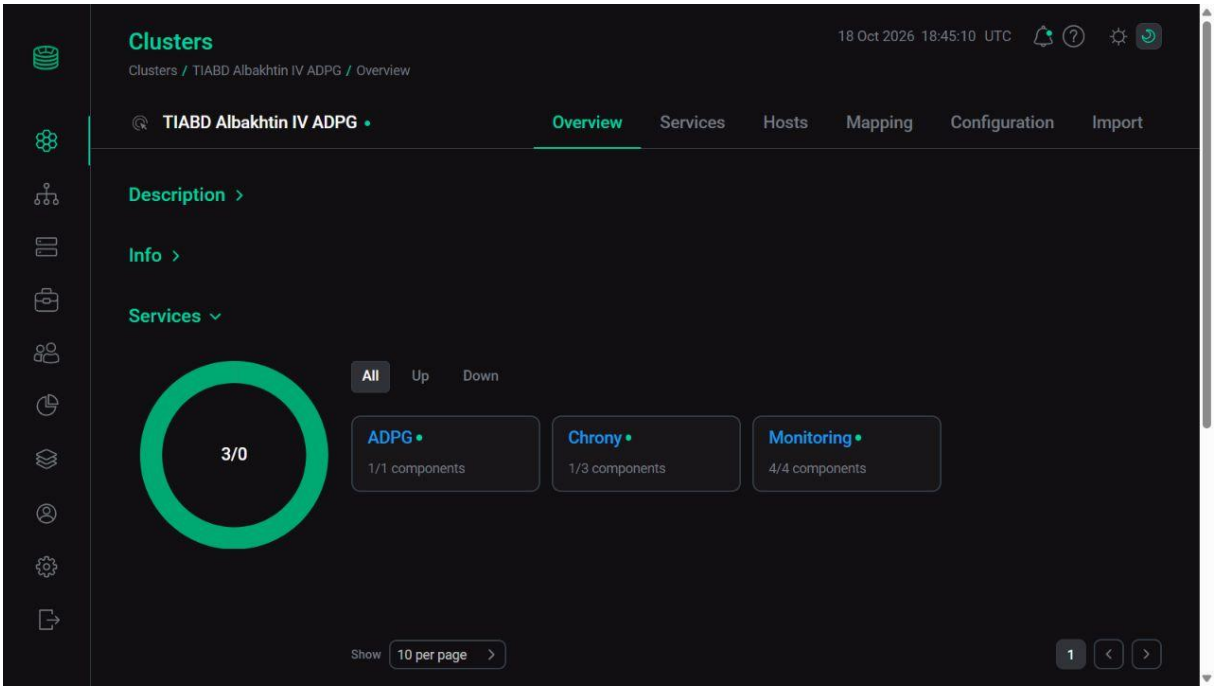


Рисунок 9 – Страница Cluster Overview со статусами служб официального кластера

Результаты приведены в таблице 7.

Таблица 7 – Доступ к сервисам

Сервис	Адрес с Windows
ADCM	http://127.0.0.1:8001
ADPG	127.0.0.1:5433; ecommerce
Prometheus	http://192.168.56.103:11200
Grafana	http://127.0.0.1:11211

13 Схема и импорт данных

Сначала созданы таблицы public, первичные и внешние ключи, затем импортированы CSV. Данные загружены в порядке зависимостей: sellers, customers, geolocation, leads_qualified, product_category_name_translation, leads_closed, products, orders, order_items, order_reviews, order_payments. В методичке leads_closed повторяется в перечне; повторный импорт одной таблицы не выполняется, а leads_qualified необходима как родительская таблица маркетинговых лидов.

Исходные CSV сохранены без изменения. Для двух отсутствующих категорий добавлены записи словаря с NULL-переводом. Ссылки leads_closed на продавцов, отсутствующих в sellers, заменены на NULL при переносе из

staging-таблицы. Остальные значения и число строк сохранены. Загрузка схемы и данных выполнена в одной транзакции с ON_ERROR_STOP; после импорта выполнен ANALYZE.

```
sudo -u postgres /usr/lib/adpg16/bin/psql -h /tmp -d ecommerce \
  -v ON_ERROR_STOP=1 -1 -f schema.sql -f load.sql
```

Результаты приведены в таблице 8.

Таблица 8 – Количество импортированных строк

Таблица	**Строк**
customers	99 441
geolocation	1 000 163
leads_closed	842
leads_qualified	8 000
order_items	112 650
order_payments	103 886
order_reviews	100 000
orders	99 441
product_category_name_translation	73
products	32 951
sellers	3 095
Итого	1 560 542

Проверены 11 первичных и 9 внешних ключей; все внешние ключи имеют convalidated=true. Схема получена непосредственно из системного каталога работающей базы командой show_schema.sql. Геолокация содержит несколько строк на почтовый индекс, поэтому индекс почты не используется как уникальный родительский ключ.

На рисунке 10 показан результат выполнения на стенде.

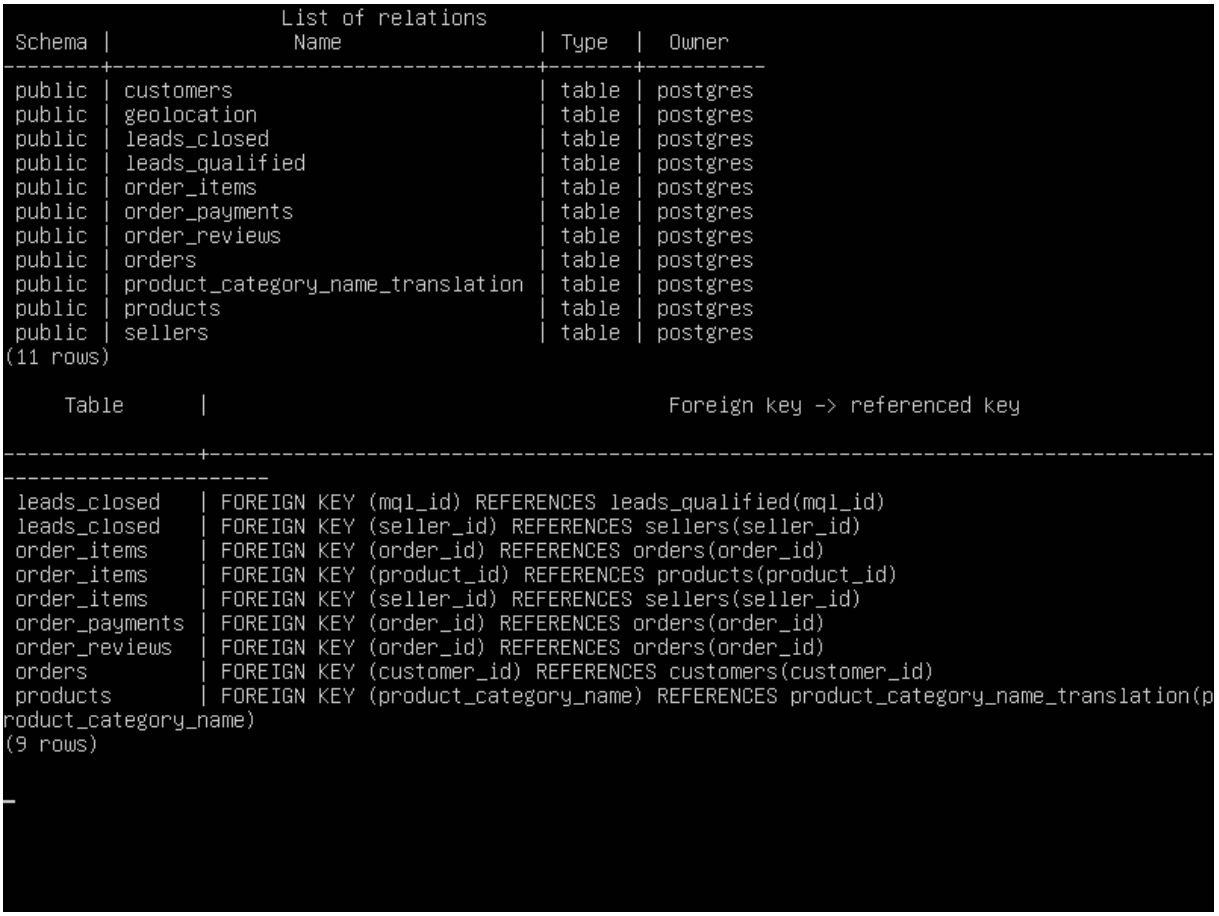


Рисунок 10 – Таблицы и связи public-схемы в консоли psql установленного ADPG

14 Аналитические SQL-задания

14.1 Топ категорий по выручке

```
-- 1. Top 10 categories by revenue with English translation.
SELECT
  p.product_category_name,
  COALESCE(t.product_category_name_english, 'not translated') AS category_en,
  ROUND(SUM(oi.price), 2) AS revenue,
  COUNT(DISTINCT oi.order_id) AS orders_count,
  COUNT(*) AS items_count
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
LEFT JOIN product_category_name_translation t
  ON t.product_category_name = p.product_category_name
GROUP BY p.product_category_name, t.product_category_name_english
ORDER BY revenue DESC
LIMIT 10;
```

Результаты приведены в таблице 9.

Таблица 9 – Топ категорий по выручке

Категория	Перевод	Выручка R$	Заказы	Позиции
beleza_saude	health_beauty	1 258 681,34	8 836	9 670
relogios_presentes	watches_gifts	1 205 005,68	5 624	5 991
cama_mesa_banho	bed_bath_table	1 036 988,68	9 417	11 115
esporte_lazer	sports_leisure	988 048,97	7 720	8 641
informatica_acessorios	computers_ accessories	911 954,32	6 689	7 827
moveis_decoracao	furniture_decor	729 762,49	6 449	8 334
cool_stuff	cool_stuff	635 290,85	3 632	3 796
utilidades_domesticas	housewares	632 248,66	5 884	6 964
automotivo	auto	592 720,11	3 897	4 235
ferramentas_jardim	garden_tools	485 256,46	3 518	4 347

Лидирует категория health_beauty: 1 258 681,34 R$. Выручка определяется как SUM(price), без доставки. Заказы считаются по уникальному order_id, позиции — по строкам order_items.

14.2 Платёжное поведение

```
-- 2. Payment behaviour.
SELECT
  payment_type,
  COUNT(*) AS payments_count,
  ROUND(AVG(payment_installments), 2) AS avg_installments,
  ROUND(AVG(payment_value), 2) AS avg_payment,
  ROUND(100.0 * COUNT(DISTINCT order_id)
    / (SELECT COUNT(DISTINCT order_id) FROM order_payments), 2) AS order_share_pct
FROM order_payments
GROUP BY payment_type
ORDER BY payments_count DESC;
```

Результаты приведены в таблице 10.

Таблица 10 – Платёжное поведение

Тип	Платежи	Рассрочки	Средний R$	Заказы %
credit_card	76 795	3,51	163,32	76,94
boleto	19 784	1,00	145,03	19,90
voucher	5 775	1,00	65,70	3,89
debit_card	1 529	1,00	142,57	1,54
not_defined	3	1,00	0,00	0,00

Самый распространённый способ оплаты — credit_card. Знаменатель доли — число уникальных заказов с известной оплатой. Один заказ может использовать несколько способов, поэтому сумма долей не обязана равняться 100 %. Число рассрочек усредняется по платёжным записям.

14.3 Оценка и скорость доставки

```
-- 3. Review score depending on delivery speed.
WITH reviews AS (
  SELECT order_id, AVG(review_score) AS review_score
  FROM order_reviews
  GROUP BY order_id
), delivery AS (
  SELECT
    o.order_id,
    CASE
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 5
THEN 'up to 5 days'
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 10
THEN '6-10 days'
      ELSE 'more than 10 days'
    END AS delivery_group,
    CASE
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 5
THEN 1
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 10
THEN 2
      ELSE 3
    END AS sort_order,
    r.review_score
  FROM orders o
  JOIN reviews r ON r.order_id = o.order_id
  WHERE o.order_delivered_customer_date IS NOT NULL
)
SELECT delivery_group,
       COUNT(DISTINCT order_id) AS orders_count,
       ROUND(AVG(review_score), 2) AS avg_review_score
FROM delivery
GROUP BY delivery_group, sort_order
ORDER BY sort_order;
```

Результаты приведены в таблице 11.

Таблица 11 – Оценка и скорость доставки

Интервал	Заказы	Средняя оценка
up to 5 days	16 707	4,43
6-10 days	33 161	4,34
more than 10 days	46 608	3,90

Длительность рассчитана как разность календарных дат доставки и покупки. Отзывы усреднены до одного значения на заказ, чтобы несколько отзывов не увеличивали его вес. Заказы без даты доставки исключены. При увеличении длительности средняя оценка снижается; наблюдаемая связь не доказывает причинность.

14.4 Эффективность продавцов

```
-- 4. Top 10 sellers by number of orders.
SELECT
  oi.seller_id,
  COUNT(DISTINCT oi.order_id) AS orders_count,
  COUNT(DISTINCT c.customer_unique_id) AS customers_count,
  COUNT(*) AS items_count,
  ROUND(SUM(oi.price), 2) AS revenue,
  ROUND(AVG(oi.freight_value), 2) AS avg_freight
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY oi.seller_id
ORDER BY orders_count DESC, revenue DESC
LIMIT 10;
```

Результаты приведены в таблице 12.

Таблица 12 – Эффективность продавцов

Продавец	Заказы	Клиенты	Позиции	Оборот R$	Доставка R$
6560211a	1 854	1 824	2 033	123 304,83	13,75
4a3ca931	1 806	1 792	1 987	200 472,92	17,65
cc419e06	1 706	1 657	1 775	104 288,42	14,46
1f50f920	1 404	1 388	1 931	106 939,21	18,21
da8622b1	1 314	1 275	1 551	160 236,57	16,09
955fee92	1 287	1 282	1 499	135 171,70	16,97
7a67c85e	1 160	1 155	1 171	141 745,53	17,85
ea8482cd	1 146	1 133	1 203	37 177,52	14,58
4869f7a5	1 132	1 124	1 156	229 472,63	17,45
3d871de0	1 080	1 074	1 147	94 914,20	19,56

В таблице показаны первые восемь символов seller_id; полные идентификаторы сохранены в storage.json. Покупатели считаются по customer_unique_id. Последний столбец содержит среднее значение freight_value на товарную позицию. Рейтинг отсортирован по числу уникальных заказов, затем по обороту.

15 Визуализация в Grafana

Создан источник PostgreSQL ecommerce-postgres с подключением к установленному ADPG и пользователем только для чтения public-схемы. Проверка источника возвращает OK. Дашборд включает месячную динамику уникальных заказов, распределение заказов по оценкам, способы оплаты, топ-

10 категорий по обороту и две дополнительные панели: суммарную стоимость товарных позиций и число уникальных покупателей.

На рисунке 11 показан результат выполнения на стенде.

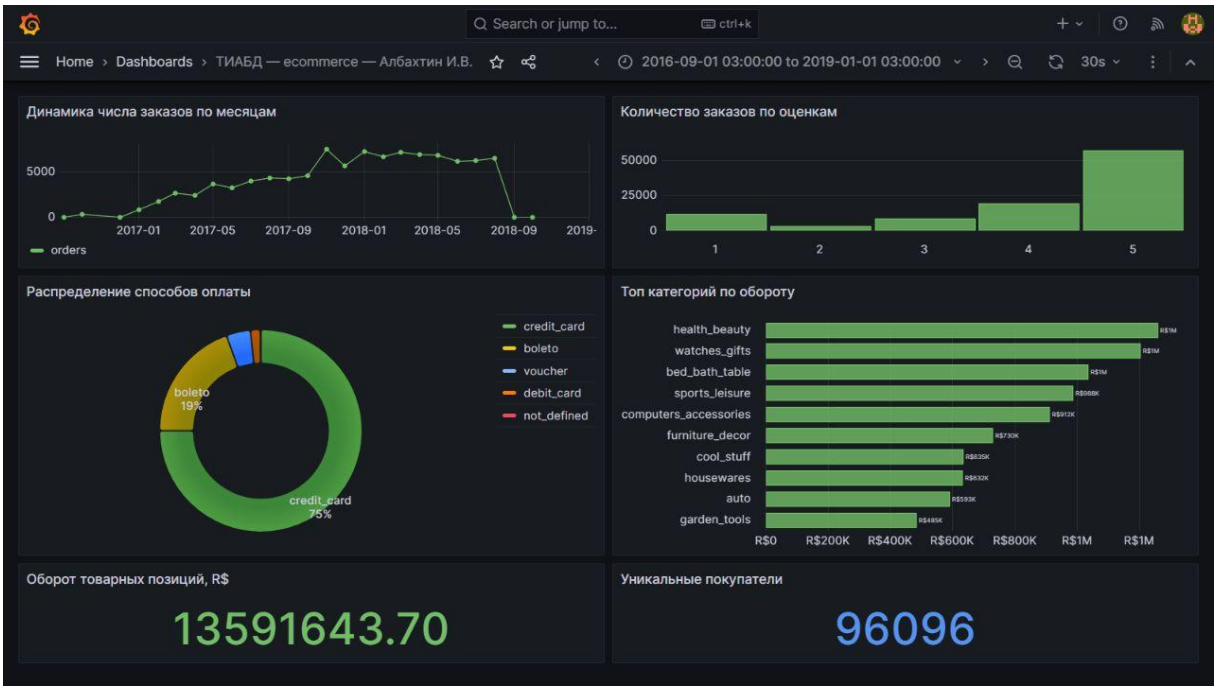


Рисунок 11 – Общий дашборд ecommerce с шестью панелями в Grafana

На рисунке 12 показан результат выполнения на стенде.

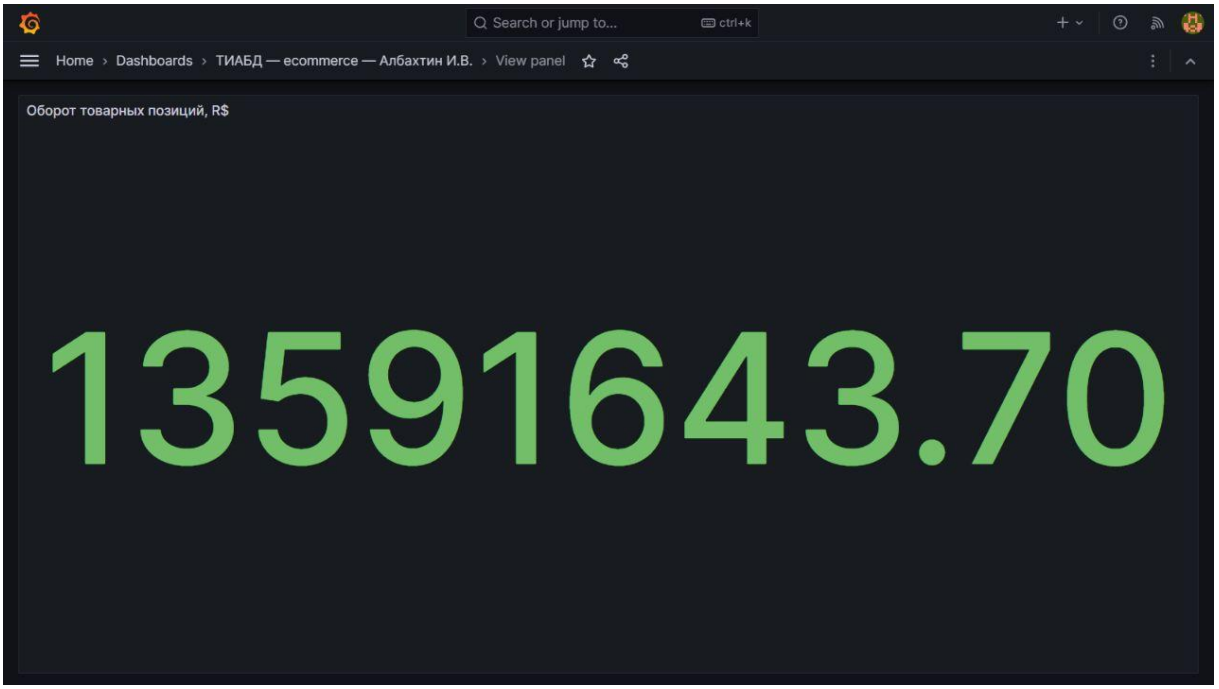


Рисунок 12 – Дополнительная визуализация оборота товарных позиций

На рисунке 13 показан результат выполнения на стенде.

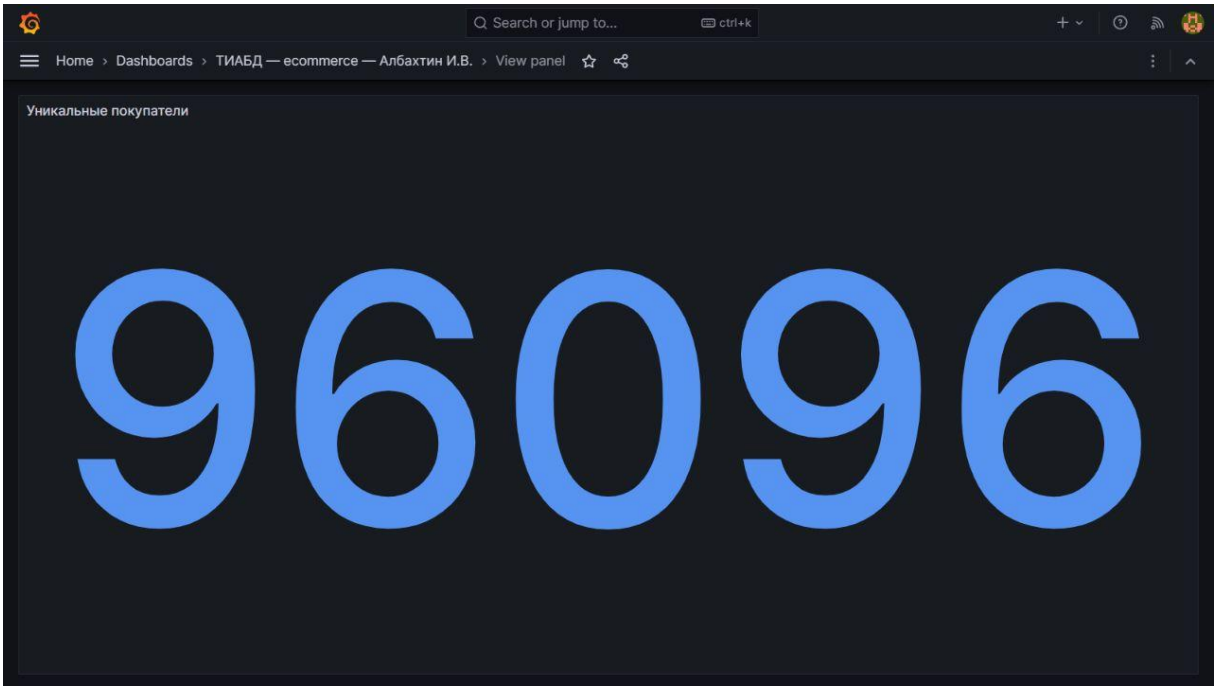


Рисунок 13 – Дополнительная визуализация числа уникальных покупателей

Результаты приведены в таблице 13.

Таблица 13 – Контрольные показатели

Показатель	**Значение**
Оборот товарных позиций	13 591 643,70 R$
Уникальные покупатели	96 096
Grafana datasource health	OK
Доступность целей Prometheus	3 из 3; up = 1

Линейный график показывает изменение числа заказов по месяцам. Наиболее многочисленна оценка 5; при чтении столбцов учитывается число уникальных заказов для каждой оценки. Круговая диаграмма отображает распределение вхождений способов оплаты в заказы. Горизонтальные столбцы позволяют сравнить оборот категорий. Дополнительные показатели описывают общий масштаб набора данных.

16 Выводы

Официальный кластер Arenadata развёрнут через ADCM на Ubuntu Server с двумя сетевыми адаптерами. Установлены пакеты ADPG, настроены мониторинг и источник Grafana. В ecommerce загружены 11 связанных таблиц; целостность подтверждена первичными и внешними ключами. Выполнены четыре SQL-задания и построен дашборд из шести панелей. Полученные

результаты показывают ведущие категории и способы оплаты, различия активности продавцов и снижение средней оценки при более длительной доставке.

17 Ответы на контрольные вопросы по хранению данных

1. Почему создание таблиц должно выполняться до импорта данных из CSV-файлов?

Команда импорта PostgreSQL загружает строки в уже существующую таблицу. До загрузки нужно определить имена и типы столбцов, первичные и внешние ключи, ограничения NOT NULL и индексы, необходимые для дальнейшей работы. Тогда СУБД проверяет данные при импорте, а ошибки видны сразу. В ecommerce, например, сначала создаются customers и orders, затем загружаются их CSV в порядке зависимостей. Для очень больших загрузок ограничения иногда добавляют после этапа staging, но и в таком процессе целевая схема должна быть определена и проверена до использования данных в аналитике.

2. Какие проблемы могут возникнуть, если структура CSV-файла не соответствует структуре целевой таблицы?

Если не совпадают порядок или число столбцов, значения попадут не в те поля либо импорт завершится ошибкой. Несовместимый тип даст ошибку преобразования: например, текст в числовом payment_value или дата в неподходящем формате. Также возможны нарушения NOT NULL, уникальности и внешних ключей. Даже успешный импорт может быть неверным по смыслу, если два текстовых столбца поменялись местами. Поэтому перед загрузкой сверяют заголовок и набор полей, разделитель, кавычки, кодировку, десятичный формат, представление пустых значений и выборочно проверяют загруженные строки и их количество.

3. Что произойдёт при попытке загрузить строку, содержащую внешний ключ, если соответствующая запись в родительской таблице отсутствует?

При действующем ограничении FOREIGN KEY PostgreSQL отклонит такую строку с ошибкой ссылочной целостности. Например, orders.customer_id не может ссылаться на покупателя, которого нет в customers. При загрузке через COPY ошибка обычно прерывает текущую команду; если импорт выполняется внутри транзакции, её изменения можно откатить. Правильное решение — сначала загрузить родительскую таблицу, а если ключа действительно нет в исходных данных, исследовать причину и явно описать способ обработки. Нельзя молча отключить внешний ключ и объявить загрузку корректной.

4. Почему перед импортом нескольких взаимосвязанных таблиц необходимо определить зависимости между ними?

Порядок загрузки должен учитывать ссылки: сначала таблицы, на которые ссылаются другие, затем зависимые таблицы. В Olist customers нужно загрузить до orders, а orders, products и sellers — до order_items; таблица переводов категорий нужна до products, если связь проверяется внешним ключом. Такой порядок предотвращает ошибки и делает импорт повторяемым. Если исходный набор содержит циклические зависимости или отсутствующие справочные записи, нужен отдельный staging-этап, проверка и осознанное разрешение этих случаев.

5. Для чего используется GROUP BY при решении аналитических задач практической работы?

GROUP BY собирает строки с одинаковым значением выбранного признака в группу, после чего агрегатные функции считают показатель для каждой группы. Так получают выручку по товарным категориям (SUM(price)), число платежей по способам оплаты (COUNT(*)), среднюю оценку по интервалам доставки (AVG(review_score)) и показатели по продавцам. Без

группировки агрегат относится ко всему набору строк и не даёт разбивки. Перед запросом важно определить уровень детализации: если соединение размножило строки платежей или отзывов, агрегаты могут оказаться завышенными.

6. Почему при подсчёте числа заказов иногда необходимо использовать COUNT(DISTINCT order_id), а не COUNT(*)?

COUNT(*) считает строки результата. В order_items один заказ может иметь несколько товарных позиций, а в order_payments — несколько платёжных строк; после JOIN строк может стать ещё больше. Если нужен показатель «сколько заказов было», используют COUNT(DISTINCT order_id). Например, заказ с тремя позициями даёт COUNT(*) = 3, но COUNT(DISTINCT order_id) = 1. Для числа позиций или платежей, напротив, подсчёт строк уместен. При сравнении долей способов оплаты нужно заранее определить знаменатель: в методичке это уникальные заказы с известными платежами.

7. Для чего используется сортировка ORDER BY ... DESC при построении рейтингов?

DESC располагает значения выбранной метрики от большего к меньшему. Это нужно, чтобы первые строки были лидерами, а LIMIT 10 действительно вернул топ-10. Для рейтинга категорий сортируют по суммарной выручке, для рейтинга продавцов — по указанному в задаче бизнес-показателю, например числу уникальных заказов. Если значения равны, полезно добавить второй ключ сортировки: ORDER BY revenue DESC, product_category_name ASC. Так результат становится стабильным и понятно, почему объекты с одинаковой выручкой стоят в данном порядке.

8. Что означает «источник данных» в Grafana и какую информацию необходимо указать для подключения PostgreSQL?

Источник данных (data source) — настройка соединения, через которую Grafana отправляет запросы в PostgreSQL и получает данные для панелей. Обычно указывают адрес и порт сервера, имя базы ecommerce, пользователя и

пароль, а также режим TLS/SSL и необходимые параметры совместимости. Адрес должен быть доступен из процесса Grafana: если Grafana и PostgreSQL работают в разных контейнерах одной сети Compose, localhost внутри Grafana означает сам контейнер Grafana, а не сервер базы. После настройки проверяют соединение и отдельно выполняют запрос панели: успешный тест соединения ещё не гарантирует, что у пользователя есть доступ к нужным таблицам и что запрос возвращает данные.

9. Почему для динамики количества заказов целесообразно использовать линейный график, а не круговую диаграмму?

Динамика — это последовательность значений во времени. Линия с месяцами по оси X показывает порядок периодов, рост, падение и возможные сезонные колебания числа заказов. Круговая диаграмма показывает доли категорий в одном целом и не позволяет удобно увидеть временной тренд; при большом числе месяцев она становится трудно читаемой. Для графика из методички сначала группируют заказы по месяцу даты оформления, считают уникальные order_id и сортируют месяцы хронологически. Ось и подпись должны ясно говорить, что показано именно число заказов, а не число товарных позиций.

10. Чем преобразование данных в Grafana отличается от преобразования данных непосредственно в SQL-запросе?

SQL-запрос обрабатывает данные на стороне PostgreSQL до передачи результата в Grafana: фильтрует строки, соединяет таблицы, группирует и считает показатели. Преобразования Grafana работают с результатом, который уже вернулся из источника; они удобны для настройки конкретной панели — переименования полей, изменения отображения, объединения небольших наборов. Крупные соединения и расчёт бизнес-метрик обычно лучше делать в SQL, чтобы не передавать лишние строки и иметь одинаковое определение показателя для разных панелей. Преобразование в Grafana не исправит ошибочную детализацию исходного SQL.

11. Что может означать отсутствие данных на панели Grafana при успешно установленном подключении к PostgreSQL?

Соединение проверяет доступность базы, но панель зависит от собственного запроса и настроек. Возможные причины: выбранный временной диапазон не включает строки, SQL-фильтр отсекает всё, задана неверная переменная дашборда, у пользователя нет доступа к таблице, столбец времени содержит NULL или формат результата не подходит типу панели. Проверка начинается с SQL-запроса без дополнительных фильтров, затем с Query inspector, выбранного диапазона времени и параметров панели. Для месячной динамики также нужно убедиться, что поле месяца распознано как время, а запрос возвращает числовой показатель.

12. Почему визуализация должна сопровождаться текстовой интерпретацией результата, а не только изображением графика?

График показывает данные, но сам не формулирует ответ на задачу: какие группы лидируют, в каких единицах измерена метрика, за какой период она рассчитана и какие ограничения есть у сравнения. Подпись и короткий вывод превращают изображение в проверяемый результат. Например, по графику сроков доставки можно сказать, что в сохранённой выборке средняя оценка ниже при более долгой доставке, но это наблюдаемая связь, а не доказательство, что задержка сама по себе вызвала снижение оценки. Текст также помогает заметить ошибки: доли способов оплаты могут суммироваться более чем в 100 %, если один заказ использовал несколько способов, и это надо объяснить.

18 Источники и приложения

1. Практическая работа №2 «Система хранения данных». Предоставленная методичка, разделы 5 и 6.

2. Официальные бандлы и репозитории Arenadata: https://downloads.arenadata.io/bundles/;

https://docs.arenadata.io/en/ADPG/current/get-started/online-install/adpg-install/cluster-creation.html. Дата обращения: 18.10.2026.

3. Brazilian E-Commerce Public Dataset by Olist. Исходные CSV и их контрольные суммы сохранены в личном стенде; реестр source_inputs.json находится рядом с материалами подготовки данных.

4. Приложения: Исходники/ПР2_Хранение — SQL, официальный манифест бандлов и команды повторного запуска; Результаты/Хранение — storage.json, adcm.json и снимки фактического выполнения.